Group 42 : Negotiating Agent

Anna Głodek , **Mete Aksoy** , **Simona Simeonova** and **Ananya Singh**

1 Introduction

Negotiation is a process through which players, on their own, try to reach an agreement. This can be a time consuming process and agreeing on a deal is often a tough challenge for human negotiators. To overcome these limitations, this paper aims to explore automated negotiation in which agents assist humans in the decision making process [Weiss and Chen, 2018].

In this paper, we are exploring bi-lateral negotiation with agents that have conflicting interests and are trying to reach an agreement. We will be discussing the impact of these agents on single issue and multi-issue domains, in scenarios with varying weightings of issues and evaluate these agents in a tournament.

We will be using the NegMAS framework for designing and implementing these agents, with the SAOP (Stacked Alternating Offers Protocol). The setup will entail two agents with conflicting interests, that have no opponent preference knowledge, trying to arrive to an agreement on varying issues.

We will first start my introducing the different domains that will be used to test our agent, followed by implementing our basic agent. This will be used as the benchmark to compare our other agent to. Then, we will be creating more agents by improving on our current agent, trying out a new strategy and trying to combine different strategies together. For every agent we will describe the structure, the negotiation strategy implemented and evaluate it against other agents. In the end, we will do a final evaluation in a tournament style competition and finally end with a conclusion.

2 Domains

We will be experimenting with 5 scenarios building on difficulty in the context of buying and selling houses.

First, we start with a single issue of price. In scenario 2, we add a second issue of location where the buyer and seller equally care about the location and price. In scenario 3, we will introduce weights where the buyer cares about price only slightly more than about location, and the seller cares mostly about price. Then, we will introduce two multi-issue scenarios where price, location, energy label, size, and property conditions of the house need to be considered. At first, the buyer and seller will care equally about all issues. Then, the buyer will care mostly about price and location, while the seller will still care about price the most. An overview of the precise issues and weights is found in Appendix A.

3 Creating The Base Agent

3.1 Agent and its Structure

The base negotiation agent we started with is time-based with a linear concession function. It is designed using the NegMAS framework and inheriths from the *SAONegotiator* class. The key methods of the agent are listed below.

- *__init__(self, gamma=1.0, reserved_value, **kwargs)*: The constructor initializes the agent and sets the concession speed through parameter *gamma*. A higher value means the agent concedes slower and vice versa.
- *propose(self, state)*: This method generates an offer based on the agent's utility function and the current negotiation time. The target utility is calculated according to a linear concession schedule. The offer with utility just above the target is selected to avoid overdemanding. If there is no such offer, a random outcome is proposed.
- *respond(self, state)*: This method evaluates the incoming offers from the opponent. It computes the time-dependent target utility and accepts any offer that meets or exceeds that threshold.
- *on_preferences_changed(self, changes)*: This method is automatically called when the agent's preferences change and it included solely for purposes of speeding up the evaluation tests.
- *_all_outcomes(self)*: This method pre-computes all possible deals once preferences are set and is also included for speed-up purposes in order to not recompute all combinations everytime it wants to propose an offer.

The agent uses a linear additive utility function to evaluate each outcome. It also has a reservation value representing the minimum utility it is willing to accept.

Overall, the base agent is modular by separating the proposal and response logic, making it a good starting point for future enhancement.

3.2 Negotiation Strategy

The base agent uses a linear concession approach for its negotiation strategy. It gradually reduces its utility demand as negotiation progresses. This time-dependent strategy can balance assertiveness earlier in the negotiation and flexibility closer to the deadline.

The agent calculates its target utility at each step using the formula $U_{\text{target}} = U_{\max} - (U_{\max} - U_{\min})\, t^{\gamma}$, where t is the normalized time, γ is the concession speed parameter, U_{max} is the maximum utility of 1.0, and U_{min} is the reservation utility. By using this formula, the agent starts demanding higher utility and decreases gradually over time.

When proposing an offer, the agent searches the outcome space and selects the one with closest to target utility without going below. This ensures the agent does not over-demand. The acceptance rule of the agent is to accept any offer that meets or exceeds the current target utility.

To prepare for a negotiation, the negotiation space is defined by creating issues. The utility function is set up with weights for each issue and a reservation value is set.

3.3 Evaluation

Now that we have our base agent, let us run it through some tests to analyze its performance which can be found in depth in Appendix B. For the setup of the agents, to see as a sanity check we ran 2 base agents against each other for 100 runs in 30 different sessions. We used a fixed reservation value of 0.2 and gamma of 1.0 which was a reasonable estimate to let the negotiators reach an agreement but not so high that they would never reach one. The agent was able to reach an agreement with itself, so we ran the agent both as the buyer and seller against Aspiration Negotiators and Random Negotiators. We ran this experiment for the gamma values of 0.5, 1.0 and 2.0 to see how the agents work with varying concession rate.

First, we analyzed the base agent as a buyer against the aspirational negotiator as a seller. Through our experimentation, the agents always came to an agreement. Overall, the social welfare which is the sum of the average utilities of both agents increased as γ increase from 0.5 to 2.0. However, the average utilities of the aspirational negotiator acting as the seller consistently was higher than the base agent across all five scenarios, which can be seen in 1. If we increase the γ value though, the gap in average utilites between the two agents goes decrease for a fairer agreement.

Now, let us see what happens if the base agent becomes the seller and the aspirational negotiator as the buyer. Again, through out iterations, the agents came to an agreement 100% of the time. However, as seen in Figure 1 again, the aspirational negotiator outperforms the base agent in terms of utility in all scenarios.

Let us now try the base agent as a buyer against a random agent. Running the iterations this time though, the agents came to an agreement around 67% of the time. In this case, the base agent did manage to outperform the random negotiator, gaining a much higher average utility across all scenarios as seen in Figure 2. Again, switching the roles around, the agreement rate increased to 71% and the base agent still managed to outperform the random agent in terms of average utility throughout the 5 scenarios. As with the other experiment, increasing the γ value, resulted in a more balanced agreement.

Through this, we can deduce that our base agent is a good foundational negotiator since it can get a higher utility than a random negotiator across all complexities of scenarios. We can also conclude, that increasing the γ in this scenario, allows the base agent to get a more balanced agreement with the other agent (in this case higher against the aspirational agent). However, there is still room for improvement, since the aspirational negotiator was able to take advantage of our agent and get a higher utility for itself in both cases as a buyer and a seller. An advantage of this agent though, is that when negotiating with a non-random agent, an agreement was reached 100% of the time, at the cost for conceding a higher amount.

4 Improving On The Base Agent

4.1 Agent and its Structure

From the previous agent, we understood the advantages of the base agent. The base agent has a high agreement rate, however in case of more complex agents, like the aspirational negotiator, it was taken advantage of resulting in the opposing agent gaining a significantly higher utility that itself. We also found out that increasing the gamma, made the agent work against the aspirational negotiator better, hinting at the fact that the base agent was conceding too much in the beginning.

We are still using a time-based agent but now instead of a linear concession function, we will be changing the γ value dynamically. We are still using the NegMAS framework and inheriting from the *SAONegotiator* class.

As for the key methods of the negotiation agent, we have kept all methods the same except the following:

- *__init__(self, gamma=1.0, reserved_value=0.2, memory = 5, **kwargs)*: For the constructor we added the memory as a parameter, which indicated how far back in the opponent's offer we look at. Additionally, inside the constructor, we initalise a new datastructure to store the opponent's offers.
- *propose(self, state)*: This method still generates an offer based on the agent's utility function, however now it calculates the average utility of opponent offers and adjusts the γ value to it.

The rest of the methods have been kept the same as the base agent and serve the same purpose as stated in the previous chapter.

4.2 Negotiation Strategy

To first tackle the problem of conceding too early, we decided to implement a dynamic γ agent. We store all the offer made by the opponent within the agent and have an additional parameter of memory. The memory indicated how far back we will consider the opponent's offers. A memory value of 5 indicated when deciding our γ value we will take into account the last 5 offers made by the opponent.

Hence, in the propose function, we calculate the average utility of the last *memory* offers made by the opponent. If this utility is less than half of our maximum utility it means

that the opponent's offers are not desirable, hence we should concede to not reach a deadlock which means increasing the gamma value. On the contrary, if the offer is desirable to us, we do not want to concede in order to not get a worse offer, therefore we decrease the γ value.

Now, that we have adjusted our concession rate depending on what the opponent is doing, we can also tackle the offers that we return. In the base agent we return the minimum offer that is above our target. In this case however, we first get a list of all eligible offers, then calculate their probability distribution depending on the utility they offer, meaning a higher utility would have a higher chance of being picked. Finally, we propose an offer using *random.choices* to return an offer respective of the probability distribution. An explanation of the negotiation strategy, decision function for accepting offers, any important preparatory steps, and heuristics that the agent uses to decide what to do next.

4.3 Evaluation

To analyse the improved agent, we repeated the setup as described in the last chapter. First considering the improved agent versus the aspirational agent, we can see a vast improvement from the last agent, since the improved agent managed to get a higher average utility that its opponent, in every scenario as both the buyer and seller. We can also see that in Figure 3, the gap between the average utility of the two agents is much more balanced than with the old agent. As with before, the agreement rates between the two agents, in every combination and γ value was 100%. However it is important to note that for this case, increasing the γ value had negligible differences to the result.

Now, let us look at the tests between the random agent and improved agent. The base agent was able to take advantage of the random agent, and now with the improved changes, the utility obtained by the improved agent is even greater which can be seen in Figure 4. The agreement rate when the random agent was a buyer and the improved agent the seller was around 63% with the reverse resulting in around 67%, hence we can notice a decrease in the agreement rates with the random agent.

As we can see with the results above, dynamically changing the γ value as we went on and proposing statistically better offers, drastically improved the performance of our agent. The advantages of this new agent is that, it has the ability to agree of offers that obtain a higher utility for itself when negotiating with an aspirational agent and the agreements themselves are more balanced in terms of average utility when compared to the base agent. However, a disadvantage would be the drop in agreement rates between the random and improved agent attaining to the fact that it is strategically conceding. While this agent still kept some of the properties of a time-based agent, it was showing adaptive behaviors, so it would be worthwhile to try out an agent with a different strategy all together, which will be explored in the next chapter.

5 Trying Out A New Strategy

5.1 Agent and its Structure

A high-level description of the agent and its structure, including the main Python methods (mention these explicitly!) used in the negotiating agent implemented in the source code.

5.2 Negotiation Strategy

An explanation of the negotiation strategy, decision function for accepting offers, any important preparatory steps, and heuristics that the agent uses to decide what to do next.

5.3 Evaluation

A section documenting the strong and weak points of your agent, the tests you performed to analyze (and improve) the negotiation strength of your agent. You must include scores of various tests over multiple sessions that you performed. Describe how you set up the testing situation and how you used the results to modify your agent, and motivate why you choose these testing methods, metrics, or graphs.

6 Strategy Switching

6.1 Agent and its Structure

A high-level description of the agent and its structure, including the main Python methods (mention these explicitly!) used in the negotiating agent implemented in the source code.

6.2 Negotiation Strategy

An explanation of the negotiation strategy, decision function for accepting offers, any important preparatory steps, and heuristics that the agent uses to decide what to do next.

6.3 Evaluation

A section documenting the strong and weak points of your agent, the tests you performed to analyze (and improve) the negotiation strength of your agent. You must include scores of various tests over multiple sessions that you performed. Describe how you set up the testing situation and how you used the results to modify your agent, and motivate why you choose these testing methods, metrics, or graphs.

7 Evaluating The Agents

8 Conclusion

References

[Weiss and Chen, 2018] Gerhard Weiss and Siqi Chen. *Automated negotiation: An efficient approach to interaction among agents*, pages 149–177. World Scientific Publishing Company, Singapore, 2018.

A Domain Weights

Scenario Issues	w_{buyer}	w_{seller}
Single Issue: price	(1.0)	(1.0)
Double Issue: price & location	(0.5, 0.5)	(0.5, 0.5)
Double Issue: price & location	(0.6, 0.4)	(0.8, 0.2)
Multi Issue: price & location & energy label & size & property conditions	(0.2, 0.2, 0.2, 0.2, 0.2)	(0.2, 0.2, 0.2, 0.2, 0.2)
Multi Issue: price & location & energy label & size & property conditions	(0.35, 0.20, 0.15, 0.15, 0.15)	(0.50, 0.15, 0.10, 0.10, 0.15)

Table 1: Weight Distributions for Buyer and Seller Across the Domain

B Appendix: Agent Evaluation Figures

B.1 Performance Metrics for the Base Agent for all 5 scenarios

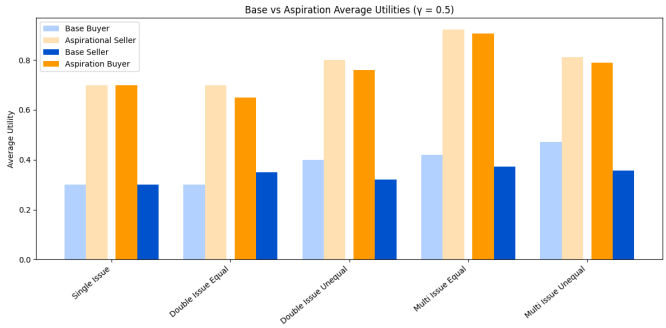

Figure 1: Average Utilities Across Scenarios for Gamma 0.5 with Aspirational and Base Agent

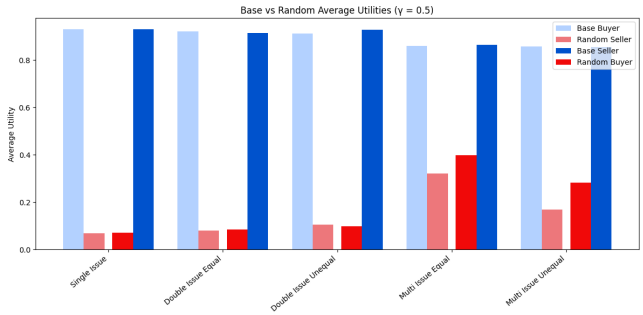

Figure 2: Average Utilities Across Scenarios for Gamma 0.5 with Random and Base Agent

B.2 Performance Metrics for the Improved Agent for all 5 scenarios

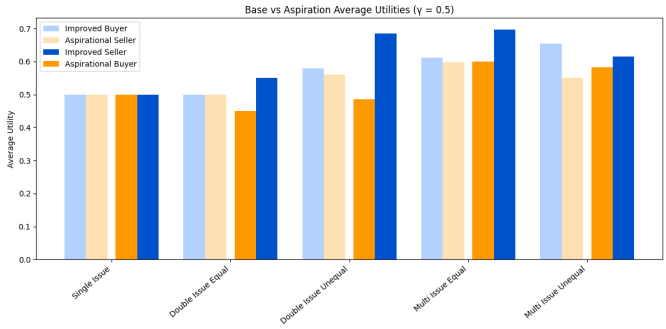

Figure 3: Average Utilities Across Scenarios for Gamma 0.5 with Aspirational and Improved Agent

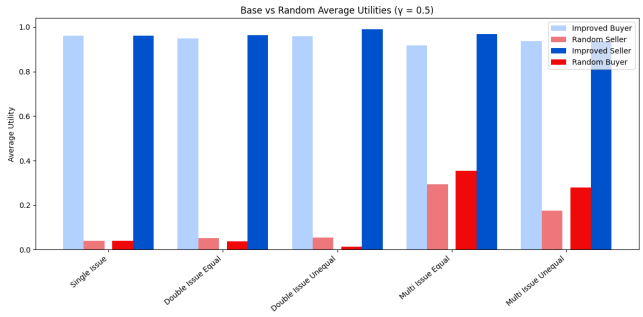

Figure 4: Average Utilities Across Scenarios for Gamma 0.5 with Random and Improved Agent